

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**  
**KHOA ĐA PHƯƠNG TIỆN**

---



**BÁO CÁO BÀI TẬP**  
**MÔN: PHÁT TRIỂN ỨNG DỤNG IOT**  
**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG**  
**PHÒNG THÔNG MINH SMARTROOM**

|                              |                      |
|------------------------------|----------------------|
| <b>Giảng viên hướng dẫn:</b> | TS. Nguyễn Quốc Uy   |
| <b>Nhóm lớp:</b>             | 01                   |
| <b>Họ và tên:</b>            | Phạm Xuân Hoàng Long |
| <b>Mã sinh viên:</b>         | B22DCPT148           |

*Hà Nội, 2026*

## **LỜI CẢM ƠN**

Em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Quốc Uy, giảng viên môn Phát triển ứng dụng IoT – Học viện Công nghệ Bưu chính Viễn thông, người đã tận tình giảng dạy, hướng dẫn và truyền đạt cho em những kiến thức nền tảng cũng như kinh nghiệm thực tiễn trong lĩnh vực Internet of Things.

Trong suốt quá trình học tập và thực hiện bài, thầy đã luôn hỗ trợ, giải đáp thắc mắc, định hướng tư duy và góp ý quý báu, giúp em hiểu rõ hơn về cách xây dựng và triển khai một hệ thống IoT hoàn chỉnh. Những kiến thức và kỹ năng mà thầy truyền đạt không chỉ giúp em hoàn thành tốt môn học mà còn là hành trang quan trọng cho quá trình học tập và làm việc sau này.

Em xin kính chúc thầy thật nhiều sức khỏe, luôn giữ vững niềm đam mê với nghề và tiếp tục gặt hái nhiều thành công trong sự nghiệp giảng dạy.

Em xin chân thành cảm ơn!

Hà Nội, tháng 3, năm 2026

Sinh viên thực hiện

Phạm Xuân Hoàng Long

## MỤC LỤC

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>DANH SÁCH CÁC HÌNH ẢNH.....</b>                                    | <b>1</b>  |
| <b>DANH SÁCH CÁC BẢNG BIỂU.....</b>                                   | <b>2</b>  |
| <b>DANH MỤC CÁC TỪ VIẾT TẮT.....</b>                                  | <b>3</b>  |
| <b>CHƯƠNG I: TỔNG QUAN.....</b>                                       | <b>4</b>  |
| I. Mục đích dự án.....                                                | 4         |
| II. Phạm vi ứng dụng.....                                             | 4         |
| III. Công nghệ sử dụng.....                                           | 5         |
| 1. Tầng thiết bị và phần cứng.....                                    | 5         |
| 2. Tầng dịch vụ và xử lý trung tâm.....                               | 5         |
| 3. Tầng giao diện người dùng.....                                     | 6         |
| <b>CHƯƠNG II: THIẾT KẾ HỆ THỐNG.....</b>                              | <b>7</b>  |
| I. Phân tích yêu cầu.....                                             | 7         |
| 1. Yêu cầu chức năng.....                                             | 7         |
| 2. Yêu cầu phi chức năng.....                                         | 7         |
| II. Tổng quan kiến trúc hệ thống.....                                 | 8         |
| 1. Mô hình kiến trúc tổng thể.....                                    | 8         |
| 2. Luồng dữ liệu trong hệ thống.....                                  | 9         |
| 3. Các thành phần công nghệ chính.....                                | 9         |
| III. Thiết kế phần cứng.....                                          | 10        |
| IV. Các use cases.....                                                | 10        |
| 1. Tác nhân.....                                                      | 10        |
| 2. Danh sách các use cases.....                                       | 11        |
| 3. Mô tả chi tiết các use cases chính.....                            | 12        |
| a. Đăng nhập.....                                                     | 12        |
| b. Hệ thống thu thập dữ liệu cảm biến và hiển thị cho người dùng..... | 12        |
| c. Người dùng điều khiển thiết bị.....                                | 14        |
| d. Xem dữ liệu cảm biến.....                                          | 16        |
| e. Luồng xem lịch sử hành động.....                                   | 18        |
| f. Quản lý hồ sơ.....                                                 | 19        |
| <b>CHƯƠNG III: XÂY DỰNG HỆ THỐNG.....</b>                             | <b>20</b> |
| I. Cơ sở dữ liệu.....                                                 | 20        |
| 1. Bảng sensors.....                                                  | 20        |
| 2. Bảng data_sensors.....                                             | 20        |
| 3. Bảng devices.....                                                  | 20        |
| 4. Bảng action_histories.....                                         | 21        |
| 5. Biểu đồ E-R.....                                                   | 22        |
| II. Backend.....                                                      | 22        |

|                                                          |           |
|----------------------------------------------------------|-----------|
| 1. Tích hợp MQTT — Giao tiếp với phần cứng.....          | 23        |
| 2. Thiết kế REST API.....                                | 24        |
| 3. WebSocket & Giao thức STOMP — Cập nhật Real-time..... | 25        |
| III. Frontend.....                                       | 25        |
| 1. Quản lý kết nối WebSocket.....                        | 25        |
| 2. Trang tổng quan.....                                  | 25        |
| 3. Trang Dữ liệu cảm biến.....                           | 26        |
| 4. Trang Lịch sử hoạt động.....                          | 26        |
| 5. Trang Hồ sơ người dùng.....                           | 27        |
| <b>CHƯƠNG IV: KẾT LUẬN VÀ ĐÁNH GIÁ.....</b>              | <b>28</b> |
| I. Kết quả đạt được.....                                 | 28        |
| 1. Kết quả.....                                          | 28        |
| 2. Một số hình ảnh kết quả đạt được.....                 | 29        |
| II. Đánh giá ưu và nhược điểm.....                       | 31        |
| 1. Ưu điểm.....                                          | 31        |
| 2. Nhược điểm.....                                       | 31        |
| III. Kết luận.....                                       | 32        |
| <b>TÀI LIỆU THAM KHẢO.....</b>                           | <b>33</b> |

## DANH SÁCH CÁC HÌNH ẢNH

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| Hình 1: Tổng quan kiến trúc hệ thống.....                                               | 8  |
| Hình 2: Thiết kế phần cứng.....                                                         | 10 |
| Hình 3: Use case tổng quan hệ thống.....                                                | 11 |
| Hình 4: Giao diện trang đăng nhập.....                                                  | 12 |
| Hình 5: Giao diện Trang tổng quan.....                                                  | 12 |
| Hình 6: Biểu đồ tuần tự luồng thu thập dữ liệu cảm biến và hiển thị cho người dùng..... | 13 |
| Hình 7: Biểu đồ tuần tự luồng điều khiển thiết bị.....                                  | 14 |
| Hình 8: Giao diện trang Dữ liệu cảm biến.....                                           | 16 |
| Hình 9: Biểu đồ tuần tự luồng xem dữ liệu cảm biến.....                                 | 16 |
| Hình 10: Giao diện trang Lịch sử hành động.....                                         | 18 |
| Hình 11: Biểu đồ tuần tự luồng xem lịch sử hành động.....                               | 18 |
| Hình 12: Giao diện trang Hồ sơ.....                                                     | 19 |
| Hình 13: Biểu đồ E-R.....                                                               | 22 |
| Hình 14: Trang tổng quan.....                                                           | 29 |
| Hình 15: Điều khiển bật đèn qua giao diện web.....                                      | 29 |
| Hình 16: Tìm kiếm dữ liệu cảm biến.....                                                 | 30 |
| Hình 17: Tìm kiếm lịch sử hành động.....                                                | 30 |
| Hình 18: Trang hồ sơ.....                                                               | 31 |

## **DANH SÁCH CÁC BẢNG BIỂU**

|                                                           |    |
|-----------------------------------------------------------|----|
| Bảng 1: Ví dụ các tham số truy vấn dữ liệu cảm biến.....  | 17 |
| Bảng 2: Ví dụ các tham số truy vấn lịch sử hành động..... | 19 |
| Bảng 3: Bảng thuộc tính quan hệ sensors.....              | 20 |
| Bảng 4: Bảng thuộc tính quan hệ data_sensors.....         | 20 |
| Bảng 5: Bảng thuộc tính quan hệ devices.....              | 21 |
| Bảng 6: Bảng thuộc tính quan hệ action_histories.....     | 21 |

## DANH MỤC CÁC TỪ VIẾT TẮT

| STT | Từ viết tắt | Ý Nghĩa                                                                               |
|-----|-------------|---------------------------------------------------------------------------------------|
| 1   | IOT         | Internet of Things: Internet Vạn Vật                                                  |
| 2   | ESP32       | Tên vi điều khiển                                                                     |
| 3   | MQTT        | Message Queuing Telemetry Transport: Giao thức truyền tin từ xa xếp hàng đợi tin nhắn |
| 4   | JSON        | JavaScript Object Notation: Kiểu dữ liệu mở trong JavaScript                          |
| 5   | API         | Application Programming Interface: Giao diện lập trình ứng dụng                       |
| 6   | E-R         | Entity-Relationship: Mô hình Quan hệ Thực thể                                         |
| 7   | MySQL       | Hệ quản trị Cơ sở dữ liệu                                                             |
| 8   | Websocket   | Giao thức truyền tin thời gian thực                                                   |

# CHƯƠNG I: TỔNG QUAN

## **I. Mục đích dự án**

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, việc ứng dụng Internet vạn vật (IoT) vào không gian sống không chỉ còn là xu hướng mà đã trở thành nhu cầu thiết thực để tối ưu hóa cuộc sống con người. Dự án "Smart Room IoT" được thực hiện với các mục tiêu chiến lược và chi tiết sau:

- Xây dựng giải pháp giám sát môi trường toàn diện: Hệ thống tập trung vào việc thu thập các dữ liệu vật lý quan trọng bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng. Những thông số này được cảm biến ghi lại và xử lý bởi vi điều khiển ESP32, sau đó truyền tải liên tục về máy chủ (Server). Việc giám sát chính xác giúp người dùng nhận biết sớm các biến đổi bất thường của môi trường, từ đó có biện pháp xử lý kịp thời để bảo vệ sức khỏe và tài sản.
- Số hóa quy trình điều khiển thiết bị: Thay vì thao tác thủ công với các công tắc vật lý truyền thống, dự án hướng tới việc "ảo hóa" quyền điều khiển. Người dùng có thể tương tác với hệ thống đèn, quạt và máy hút ẩm thông qua một giao diện web duy nhất. Điều này tạo ra sự tiện lợi tối đa, đặc biệt trong các tình huống người dùng đang ở xa hoặc quản lý nhiều phòng cùng một lúc.
- Quản lý và khai thác dữ liệu lịch sử: Một mục tiêu quan trọng khác của dự án là tính lưu vết. Mọi thay đổi về trạng thái thiết bị và mọi thông số cảm biến đều được lưu trữ vào cơ sở dữ liệu MySQL. Việc này cho phép xây dựng các báo cáo thống kê, biểu đồ phân tích xu hướng môi trường theo ngày/tuần/tháng, giúp người dùng hiểu rõ hơn về thói quen sử dụng điện năng và điều kiện môi trường trong phòng.
- Nâng cao khả năng phản hồi thời gian thực (Real-time): Dự án đặt mục tiêu giảm thiểu độ trễ trong truyền tin. Bằng việc sử dụng các giao thức tiên tiến như MQTT và WebSocket, hệ thống đảm bảo rằng khi một lệnh điều khiển được phát đi từ giao diện web, thiết bị thực tế sẽ phản ứng gần như tức thì, và ngược lại, các thay đổi môi trường được cảm biến ghi nhận sẽ được cập nhật lên màn hình người dùng trong tích tắc.

## **II. Phạm vi ứng dụng**

Hệ thống Smart Room IoT không giới hạn ở một không gian nhỏ hẹp mà có tiềm năng ứng dụng linh hoạt trong nhiều lĩnh vực đời sống và sản xuất:

- Hộ gia đình thông minh (Smart Home): Đây là phạm vi ứng dụng chính và trực quan nhất. Hệ thống có thể lắp đặt tại phòng ngủ, phòng khách hoặc phòng làm việc riêng tư. Nó giúp tự động hóa việc bật quạt khi trời nóng, bật đèn khi trời tối hoặc đơn giản là điều khiển từ xa để đảm bảo an toàn điện khi chủ nhà vắng mặt.

- Văn phòng làm việc và tòa nhà xanh: Tại các công sở, hệ thống giúp quản lý môi trường làm việc lý tưởng cho nhân viên. Việc giám sát cường độ ánh sáng giúp điều chỉnh hệ thống đèn chiếu sáng sao cho phù hợp, tránh lãng phí điện năng khi ánh sáng tự nhiên đã đủ, đóng góp vào mục tiêu xây dựng các tòa nhà tiết kiệm năng lượng.
- Lĩnh vực nông nghiệp công nghệ cao và bảo quản: Các mô hình nhà kính mini hoặc các kho lưu trữ thực phẩm hay linh kiện điện tử nhạy cảm yêu cầu sự ổn định về nhiệt độ và độ ẩm. Hệ thống có thể đóng vai trò như một trạm giám sát đáng tin cậy, giúp cảnh báo ngay lập tức nếu các điều kiện này vượt ngưỡng cho phép.
- Cơ sở giáo dục và nghiên cứu: Hệ thống có thể được sử dụng làm mô hình thực hành tại các phòng thí nghiệm IoT của các trường đại học (như PTIT). Đây là nền tảng để sinh viên nghiên cứu sâu hơn về lập trình nhúng, giao thức truyền tin và phát triển ứng dụng web hiện đại.

### **III. Công nghệ sử dụng**

Để đảm bảo tính ổn định, bảo mật và khả năng mở rộng, hệ thống được cấu thành từ những công nghệ hiện đại và phổ biến nhất hiện nay:

#### **1. Tầng thiết bị và phần cứng**

- Vi điều khiển ESP32: Được lựa chọn nhờ khả năng xử lý mạnh mẽ (Dual-core), tích hợp sẵn chuẩn kết nối Wi-Fi và Bluetooth. ESP32 vượt trội hơn các dòng Arduino cũ về bộ nhớ và tốc độ xử lý, rất phù hợp cho các bài toán truyền nhận lượng lớn dữ liệu IoT.
- Cảm biến DHT11: Một module phổ biến, tin cậy để đo đồng thời nhiệt độ và độ ẩm không khí với độ chính xác vừa đủ cho các nhu cầu dân dụng.
- Cảm biến quang trở (LDR): Dùng để thu thập giá trị cường độ ánh sáng môi trường theo thang analog, làm căn cứ để đưa ra các lệnh điều khiển đèn tự động hoặc cảnh báo.

#### **2. Tầng dịch vụ và xử lý trung tâm**

- Java 21 & Spring Boot 4: Đây là trái tim của hệ thống. Spring Boot cung cấp một khung làm việc (framework) mạnh mẽ để xây dựng các API Restful và quản lý các luồng dữ liệu phức tạp. Phiên bản Java mới nhất đảm bảo hiệu suất xử lý tốt và hỗ trợ các tính năng lập trình hiện đại.
- Giao thức MQTT (Mosquitto Broker): Đây là giao thức "vàng" trong IoT. Nó hoạt động theo mô hình Publish/Subscribe, giúp tiết kiệm băng thông và năng lượng tốt hơn nhiều so với HTTP truyền thống. Hệ thống sử dụng thư viện

Eclipse Paho và Spring Integration để thực hiện việc kết nối và điều phối tin nhắn giữa phần cứng và server.

- WebSocket & STOMP: Để giải quyết bài toán cập nhật dữ liệu liên tục lên trình duyệt mà không cần người dùng nhấn F5, WebSocket được triển khai. Giao thức STOMP chạy trên WebSocket giúp việc quản lý tin nhắn trở nên có cấu trúc và an toàn hơn.
- MySQL Database: Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, dùng để lưu trữ lâu dài các bảng dữ liệu về thiết bị, dữ liệu cảm biến, lịch sử hành động.

### **3. Tầng giao diện người dùng**

- ReactJS 19: Thư viện JavaScript hàng đầu để xây dựng giao diện người dùng dựa trên các thành phần (components). Sự linh hoạt của React giúp việc quản lý trạng thái thiết bị (on/off) trên giao diện trở nên mượt mà.
- Material UI (MUI) & Tailwind CSS: Kết hợp sức mạnh của bộ UI component chuẩn mực (MUI) và khả năng tùy biến phong cách hiện đại (Tailwind) để tạo ra một Dashboard chuyên nghiệp, hỗ trợ tốt Responsive (tương thích cả máy tính và điện thoại).
- Recharts: Thư viện chuyên dụng để vẽ bản đồ và biểu đồ. Recharts giúp chuyển đổi những con số khô khan về nhiệt độ, độ ẩm thành các đường biểu diễn trực quan, giúp người dùng dễ dàng nhận ra quy luật biến đổi môi trường.

## CHƯƠNG II: THIẾT KẾ HỆ THỐNG

### I. Phân tích yêu cầu

#### 1. Yêu cầu chức năng

Hệ thống Smart Room IoT cần đáp ứng các yêu cầu chức năng chính sau:

- Thu thập và truyền tải dữ liệu cảm biến: Hệ thống phải có khả năng đọc dữ liệu từ cảm biến nhiệt độ - độ ẩm (DHT11) và cảm biến ánh sáng (LDR) theo chu kỳ (mỗi 2 giây). Dữ liệu sau đó phải được đóng gói dưới dạng JSON và gửi lên Backend thông qua giao thức MQTT.
- Lưu trữ dữ liệu: Backend có nhiệm vụ tiếp nhận các thông điệp (messages) từ MQTT Broker, xử lý và lưu trữ vào cơ sở dữ liệu MySQL để phục vụ việc truy xuất lịch sử.
- Điều khiển thiết bị trực tuyến: Người dùng có thể thực hiện lệnh Bật/Tắt quạt (LED đỏ), đèn (LED cam) và máy hút ẩm (LED xanh) trực tiếp từ giao diện Web. Các lệnh này phải được truyền xuống thiết bị đầu cuối với độ trễ thấp nhất.
- Cập nhật trạng thái thời gian thực: Khi một thiết bị được điều khiển hoặc khi cảm biến có dữ liệu mới, thông tin phải được cập nhật ngay lập tức lên Dashboard mà không cần tải lại trang Web thông qua giao thức WebSocket.
- Báo cáo và thống kê: Dashboard phải hiển thị trực quan các thông số môi trường hiện tại, đồng thời cung cấp biểu đồ lịch sử để người dùng theo dõi sự thay đổi của nhiệt độ, độ ẩm và ánh sáng trong quá khứ.

#### 2. Yêu cầu phi chức năng

Bên cạnh các tính năng nghiệp vụ, hệ thống cần tuân thủ các tiêu chuẩn về chất lượng vận hành:

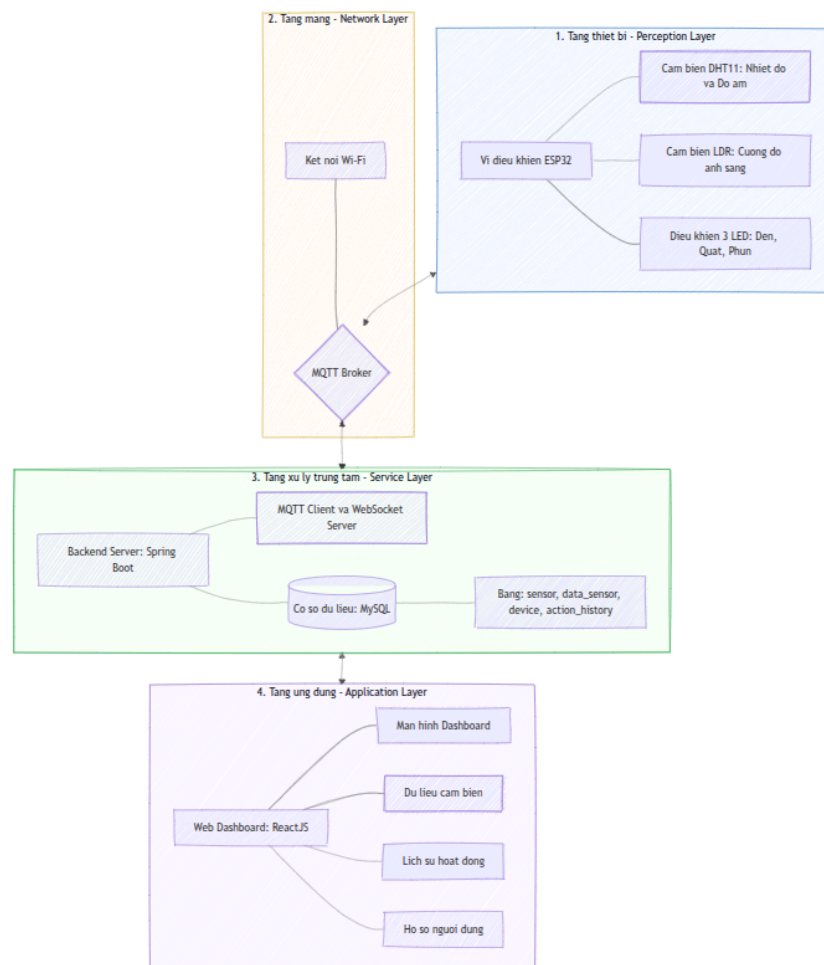
- Độ tin cậy và ổn định: Hệ thống phải có cơ chế tự động kết nối lại (auto-reconnect) khi gặp sự cố về mạng Wi-Fi hoặc MQTT Broker. Đảm bảo dữ liệu không bị thất thoát trong quá trình vận hành lâu dài.
- Hiệu năng: Thời gian phản hồi cho các thao tác điều khiển thiết bị không được quá lâu để đảm bảo trải nghiệm người dùng tốt nhất.
- Tính bảo mật: Các kết nối giữa Client (ESP32, Web) và Broker/Server phải được xác thực thông qua Username và Password. Dữ liệu nhạy cảm phải được bảo vệ tránh các truy cập trái phép.
- Tính thân thiện (UI/UX): Giao diện Dashboard phải trực quan, dễ sử dụng, hỗ trợ hiển thị tốt trên cả nền tảng Desktop và Mobile (Responsive).

## II. Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình phân tầng chuẩn IoT, giúp tách biệt trách nhiệm của từng thành phần và dễ dàng bảo trì, nâng cấp.

### 1. Mô hình kiến trúc tổng thể

Kiến trúc hệ thống bao gồm 4 tầng chính:



Hình 1: Tổng quan kiến trúc hệ thống

- Tầng thiết bị (Perception Layer): Bao gồm vi điều khiển ESP32 đóng vai trò là "bộ não" tại hiện trường, kết nối trực tiếp với các cảm biến (DHT11, LDR) và các đèn LEDs
- Tầng mạng và truyền tin (Network Layer): Sử dụng Wi-Fi và MQTT làm phương thức truyền tin chính. MQTT Broker (tương tác qua mô hình Pub/Sub) đóng vai trò trung gian điều phối dữ liệu giữa phần cứng và phần mềm.

- Tầng xử lý trung tâm (Service Layer): Được phát triển bằng Spring Boot. Tầng này chịu trách nhiệm:
  - Thực hiện vai trò MQTT Client để lắng nghe dữ liệu từ ESP32 và gửi lệnh điều khiển xuống.
  - Lưu trữ dữ liệu vào MySQL thông qua tầng Repository (Spring Data JPA).
  - Cung cấp các API RESTful cho Frontend.
  - Duy trì kết nối WebSocket để đẩy dữ liệu thời gian thực lên người dùng.
- Tầng ứng dụng (Application Layer): Dashboard xây dựng bằng ReactJS, là nơi người dùng trực tiếp giám sát môi trường và thực hiện các thao tác điều khiển thiết bị thông qua giao diện đồ họa sinh động.

## 2. Luồng dữ liệu trong hệ thống

Mô hình luồng dữ liệu của hệ thống được thiết kế theo quy trình 4 bước khép kín từ thiết bị cảm biến đến giao diện người dùng:

- Bước 1: Thu thập dữ liệu: Vi điều khiển ESP32 đọc giá trị nhiệt độ, độ ẩm từ cảm biến DHT11 và cường độ ánh sáng từ cảm biến LDR theo chu kỳ 2 giây/lần. Dữ liệu thô sau khi được xử lý sẽ được đóng gói dưới dạng định dạng JSON.
- Bước 2: Truyền tải: ESP32 đóng vai trò là một MQTT Client, thực hiện việc Publish (đăng tải) gói tin JSON lên MQTT Broker thông qua mạng Wi-Fi.
- Bước 3: Xử lý và Lưu trữ: Backend (Spring Boot) lắng nghe (Subscribe) các Topic tương ứng trên MQTT Broker. Khi nhận được dữ liệu, Backend sẽ phân tích (Parse) chuỗi JSON, kiểm tra tính hợp lệ và thực hiện lưu trữ vào cơ sở dữ liệu MySQL thông qua tầng Spring Data JPA.
- Bước 4: Hiển thị và Cập nhật:
  - Backend đồng thời đẩy dữ liệu mới nhất lên Frontend (ReactJS) thông qua kết nối WebSocket (giao thức STOMP).
  - Người dùng xem được các thông số thay đổi tức thì trên Dashboard mà không cần tải lại trang.

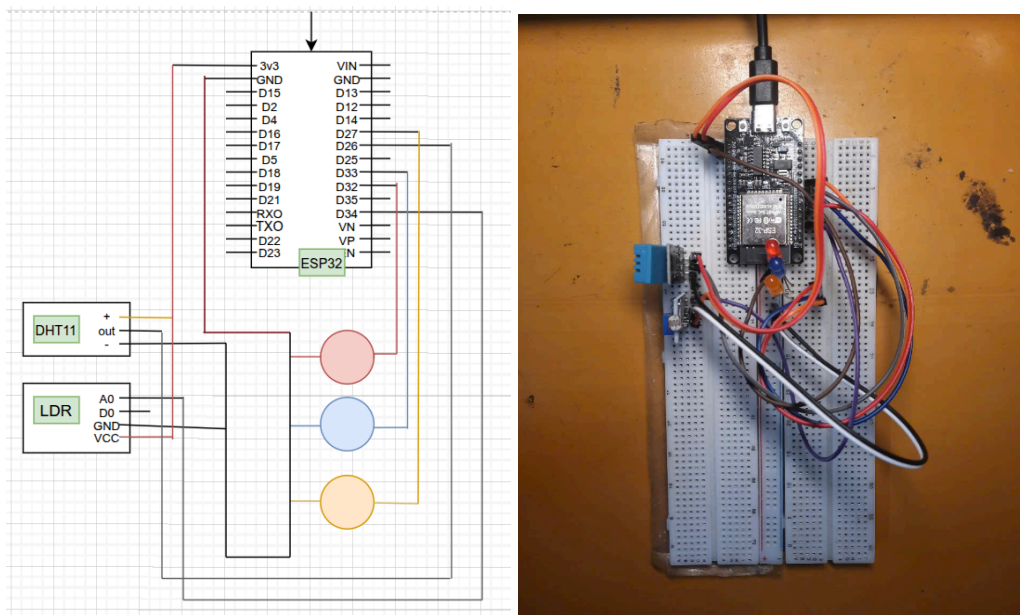
## 3. Các thành phần công nghệ chính

Hệ thống tận dụng sức mạnh của các công nghệ hiện đại để đảm bảo tính ổn định và hiệu năng:

- ESP32 (Espressif Systems): Là vi điều khiển 32-bit mạnh mẽ tích hợp sẵn Wi-Fi. Với tốc độ xử lý nhanh, hỗ trợ nhiều chân GPIO và ADC 12-bit, ESP32 là lựa chọn lý tưởng cho các ứng dụng IoT yêu cầu tính linh hoạt và tiết kiệm năng lượng.

- MQTT (Message Queuing Telemetry Transport): Giao thức truyền tin theo mô hình Publish/Subscribe cực kỳ gọn nhẹ, được thiết kế chuyên dụng cho các thiết bị IoT có băng thông thấp và môi trường mạng không ổn định.
- Spring Boot (Java Framework): Cung cấp một nền tảng Backend vững chắc với khả năng tích hợp MQTT Client và WebSocket Server một cách chuyên nghiệp, giúp việc quản lý dữ liệu và điều khiển thiết bị trở nên an toàn và hiệu quả.
- MySQL (Relational Database): Khác với các CSDL NoSQL, MySQL cung cấp cấu trúc dữ liệu chặt chẽ, hỗ trợ các truy vấn phức tạp để trích xuất lịch sử cảm biến và nhật ký điều khiển một cách chính xác.
- ReactJS (Frontend Library): Thư viện JavaScript hiện đại giúp xây dựng giao diện Dashboard mượt mà với cấu trúc Component (thành phần) dễ bảo trì, kết hợp với Recharts để trực quan hóa dữ liệu sinh động.

### III. Thiết kế phần cứng



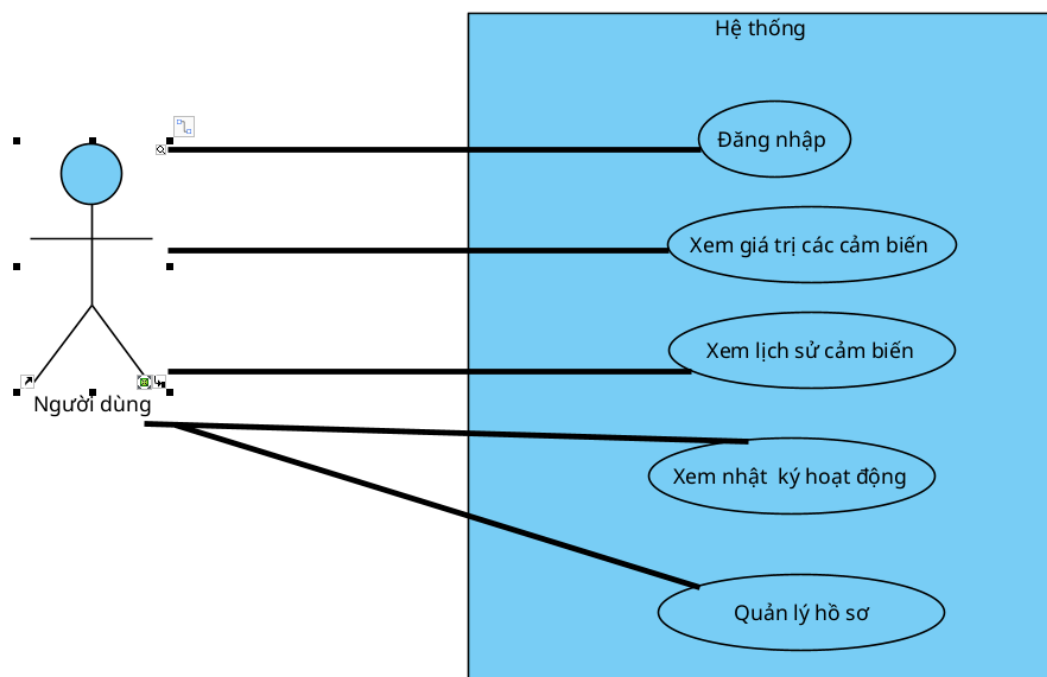
Hình 2: Thiết kế phần cứng

### IV. Các use cases

#### **1. Tác nhân**

Người dùng: Là tác nhân chính tương tác với hệ thống thông qua giao diện Web để giám sát và điều khiển thiết bị.

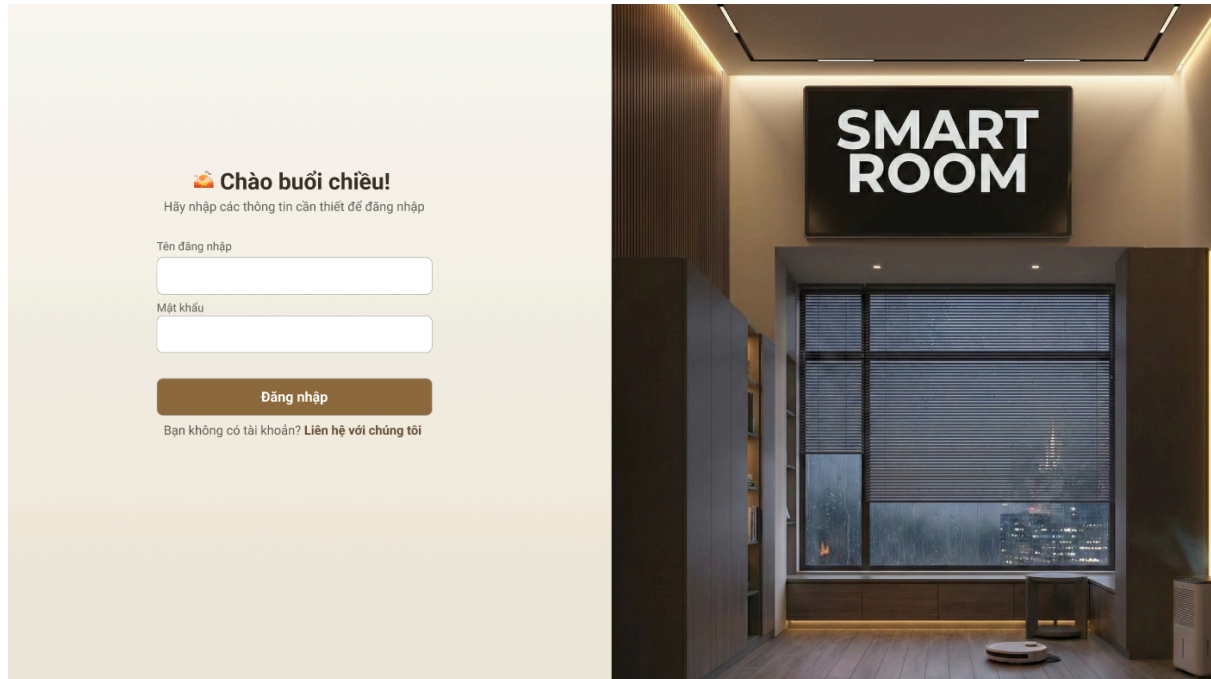
## 2. Danh sách các use cases



Hình 3: Use case tổng quan hệ thống

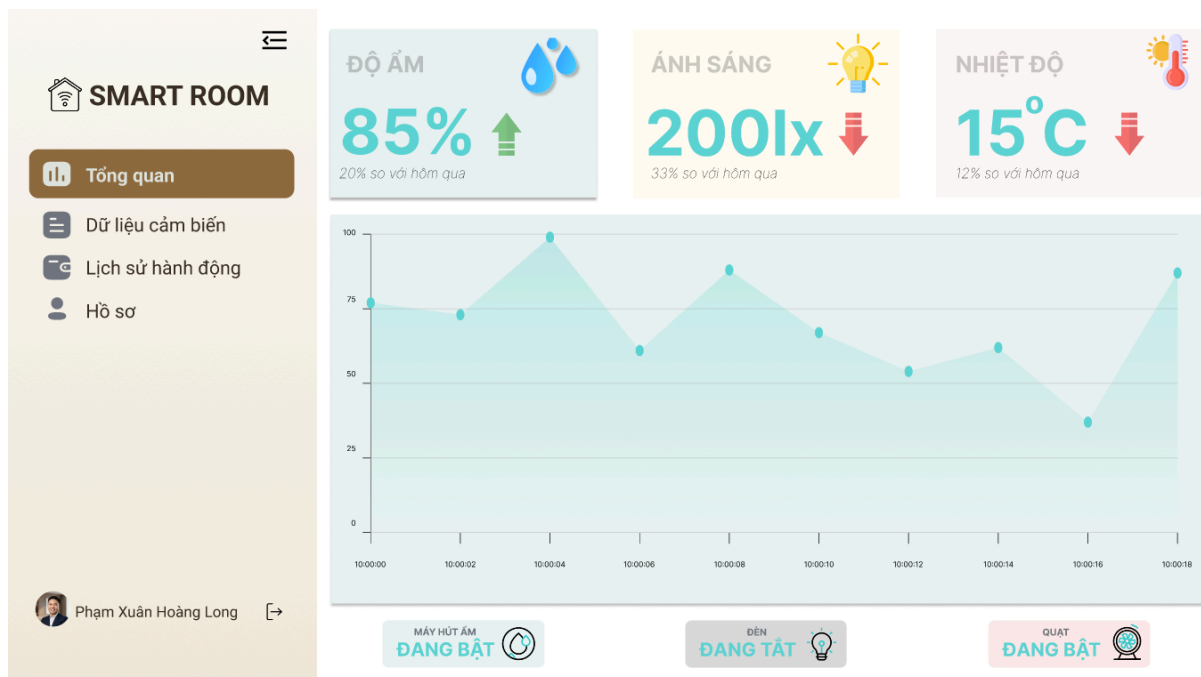
### 3. Mô tả chi tiết các use cases chính

#### a. Đăng nhập

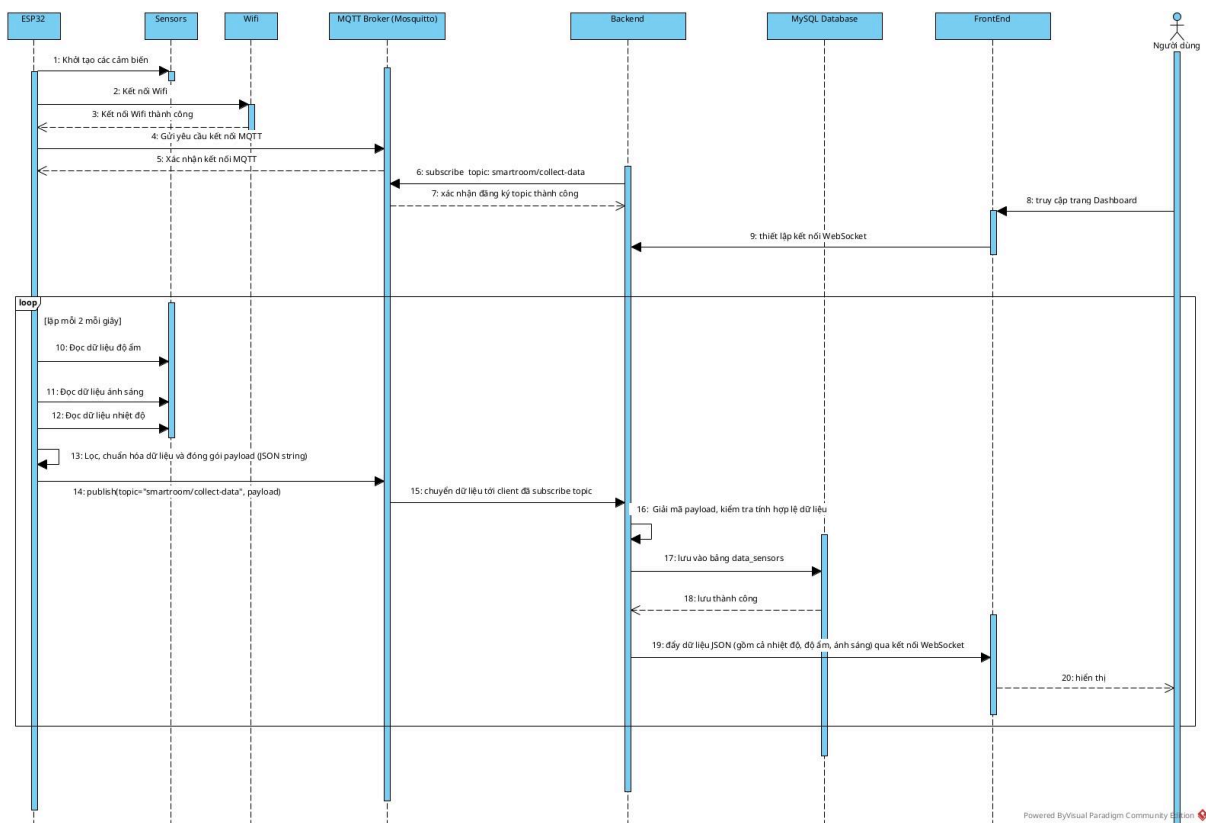


Hình 4: Giao diện trang đăng nhập

#### b. Hệ thống thu thập dữ liệu cảm biến và hiển thị cho người dùng



Hình 5: Giao diện Trang tổng quan



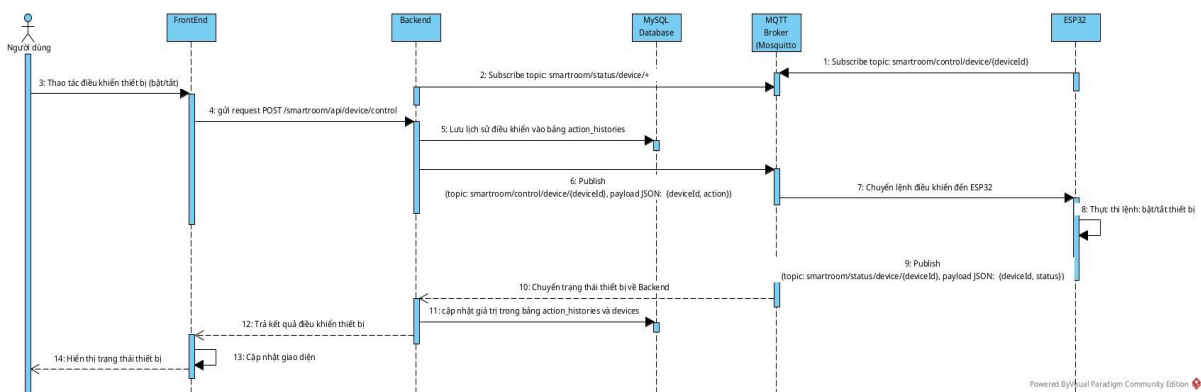
Hình 6: Biểu đồ tuần tự luồng thu thập dữ liệu cảm biến và hiển thị cho người dùng

#### Chi tiết các bước:

1. ESP32 tiến hành khởi tạo các cảm biến: Cấu hình các chân GPIO và khởi tạo cảm biến độ ẩm (DHT11), cảm biến ánh sáng (LDR) và cảm biến nhiệt độ (DHT11).
2. ESP32 gửi yêu cầu kết nối tới Wifi: Thực hiện giao thức bắt tay với Access Point.
3. ESP32 nhận thông báo kết nối Wifi thành công: Nhận địa chỉ IP động từ Router thông qua giao diện DHCP.
4. ESP32 gửi yêu cầu kết nối tới MQTT Broker (Mosquitto): Thiết lập kết nối trên tầng truyền tải TCP/IP.
5. ESP32 nhận thông báo kết nối thành công: Broker phản hồi gói tin CONNACK để xác nhận phiên làm việc.
6. Backend thực hiện đăng ký (subscribe) topic: Spring Boot đăng ký lắng nghe topic smartroom/collect-data trên Mosquitto thông qua MqttGateway.
7. MQTT Broker xác nhận đăng ký thành công: Mosquitto gửi phản hồi xác nhận quyền lắng nghe cho Backend.
8. Người dùng truy cập vào trang tổng quan: Load giao diện Dashboard từ Web Server.
9. FrontEnd thiết lập kết nối WebSocket tới Backend: Sử dụng giao thức STOMP để tạo kênh truyền dữ liệu hai chiều thời gian thực.

10. ESP32 đọc dữ liệu độ ẩm: Truy xuất giá trị kỹ thuật số từ cảm biến DHT11.
11. ESP32 đọc dữ liệu ánh sáng: Đọc tín hiệu Analog trực tiếp từ Quang trở (LDR).
12. ESP32 đọc dữ liệu nhiệt độ: Truy xuất giá trị nhiệt độ hiện tại từ cảm biến DHT11.
13. ESP32 tiến hành lọc và đóng gói JSON: Chuẩn hóa dữ liệu thành chuỗi định dạng JSON: {"temperature": 30.5, "humidity": 70, "light": 1200}.
14. ESP32 thực hiện lệnh publish: Gửi gói tin dữ liệu lên MQTT Broker theo topic đã định.
15. MQTT Broker chuyển tiếp dữ liệu tới Backend: Sử dụng cơ chế Forwarding để đẩy tin nhắn tới đối tượng đã subscribe.
16. Backend giải mã payload JSON: Sử dụng thư viện Jackson (ObjectMapper) để parse dữ liệu và kiểm tra tính hợp lệ.
17. Backend thực hiện lưu dữ liệu: Gọi SensorDataRepository để thực hiện lệnh INSERT dữ liệu vào bảng data\_sensors trong MySQL.
18. Database xác nhận lưu thành công: Đảm bảo dữ liệu đã được lưu
19. Backend đẩy dữ liệu JSON tới FrontEnd: Sử dụng SimpMessagingTemplate để truyền dữ liệu qua kết nối WebSocket đã có.
20. FrontEnd hiển thị giao diện: Cập nhật biểu đồ và các thẻ thông tin ngay lập tức cho người dùng.

### c. Người dùng điều khiển thiết bị



Hình 7: Biểu đồ tuần tự luồng điều khiển thiết bị

#### Chi tiết các bước:

1. ESP32 tiến hành đăng ký nhận lệnh tại topic smartroom/control/device/{deviceId} thông qua hàm client.subscribe().
2. Backend (Spring Boot) thực hiện đăng ký theo dõi trạng thái tại topic wildcard smartroom/status/device/+ để quản lý tập trung phản hồi từ thiết bị.

3. Người dùng nhấn nút điều khiển thiết bị (bật/tắt) trên giao diện Dashboard. Ngay lập tức, FrontEnd chuyển sang trạng thái Loading (Chờ phản hồi) để thông báo hệ thống đang giao tiếp với phần cứng.
4. FrontEnd gửi yêu cầu API POST /smartroom/api/device/control chứa lệnh điều khiển: {"deviceId": 1, "action": "ON"} tới Backend qua Axios.
5. Backend lưu thao tác này vào bảng action\_histories trong cơ sở dữ liệu MySQL để phục vụ việc truy xuất lịch sử hành động.
6. Backend publish gói tin lệnh điều khiển: {"deviceId": 1, "action": "ON"} tới MQTT Broker (Mosquitto) qua giao diện MqttGateway.
7. MQTT Broker (Mosquitto) thực hiện chuyển lệnh điều khiển đến đúng ESP32 dựa trên cơ chế Topic Matching.
8. ESP32 nhận lệnh trong hàm callback(), giải mã JSON và thực hiện xuất tín hiệu ra chân GPIO để điều khiển bật/tắt thiết bị thực tế.
9. Sau khi thực thi xong, ESP32 publish trạng thái thực tế mới: {"deviceId": 1, "status": 1} về topic smartroom/status/device/1.
10. MQTT Broker (Mosquitto) chuyển tiếp trạng thái thiết bị hiện tại về cho Backend để cập nhật hệ thống.
11. Backend nhận tin nhắn, hoàn tất cập nhật giá trị trạng thái mới nhất vào Database (bảng action\_histories và bảng devices).
12. Backend trả về kết quả điều khiển thành công dưới dạng JSON cho FrontEnd.
13. FrontEnd nhận phản hồi từ Backend, tắt trạng thái Loading và thực hiện cập nhật lại trạng thái hiển thị thực tế cho Dashboard.
14. FrontEnd hiển thị trực quan trạng thái thiết bị mới nhất cho người dùng theo dõi và nhận phản hồi thành công.

## d. Xem dữ liệu cảm biến

Tổng quan

**Dữ liệu cảm biến**

Lịch sử hành động

Hồ sơ

Phạm Xuân Hoàng Long

Tìm theo tên cảm biến  
Tất cả

Tìm theo giá trị cảm biến

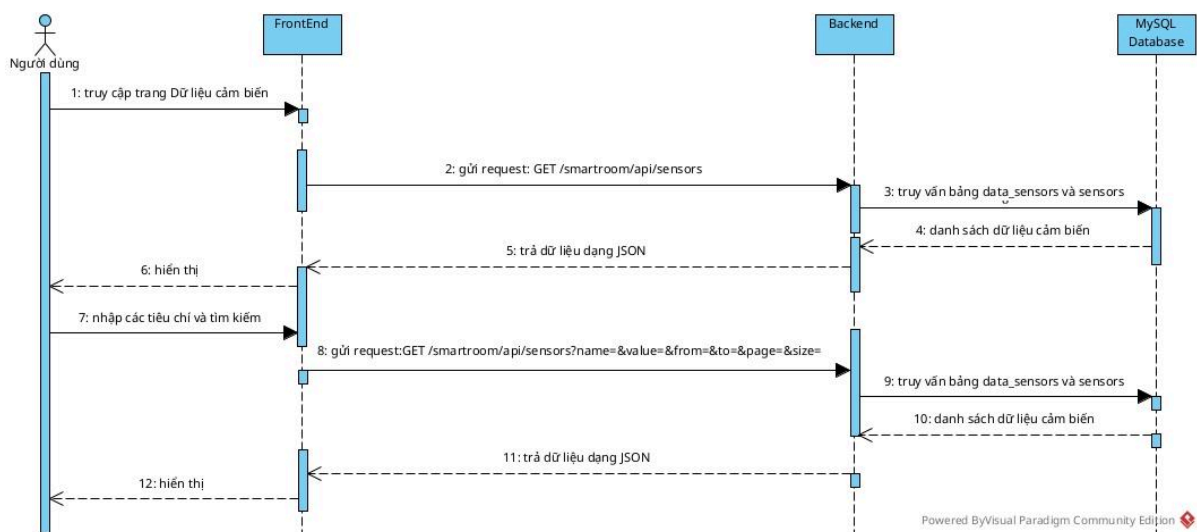
Tìm theo thời gian

Tìm kiếm

| ID     | CẢM BIẾN          | GIÁ TRỊ CẢM BIẾN | THỜI GIAN ↑↓        |
|--------|-------------------|------------------|---------------------|
| SEN001 | Cảm biến độ ẩm    | 65%              | 09:55:21 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:23 15/01/2026 |
| SEN003 | Cảm biến ánh sáng | 176lx            | 09:55:23 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:25 15/01/2026 |
| SEN003 | Cảm biến ánh sáng | 178lx            | 09:55:27 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:28 15/01/2026 |
| SEN001 | Cảm biến độ ẩm    | 65%              | 09:55:29 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:30 15/01/2026 |
| SEN003 | Cảm biến ánh sáng | 177lx            | 09:55:32 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:33 15/01/2026 |
| SEN001 | Cảm biến độ ẩm    | 63%              | 09:55:35 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:36 15/01/2026 |
| SEN003 | Cảm biến ánh sáng | 173lx            | 09:55:38 15/01/2026 |
| SEN002 | Cảm biến nhiệt độ | 25°C             | 09:55:40 15/01/2026 |
| SEN001 | Cảm biến độ ẩm    | 64%              | 09:55:41 15/01/2026 |

Trang trước
1 2 3 ... 10
Trang sau
Số hàng một trang 15

Hình 8: Giao diện trang Dữ liệu cảm biến



Hình 9: Biểu đồ tuần tự luồng xem dữ liệu cảm biến

Chi tiết các bước:

1. Người dùng truy cập vào trang Dữ liệu cảm biến
2. Frontend gửi yêu cầu api GET /smartroom/api/sensors đến Backend để lấy danh sách tất cả dữ liệu cảm biến của các thiết bị kèm phân trang mặc định
3. Backend tiếp nhận yêu cầu và thực hiện truy vấn (Join) giữa hai bảng data\_sensors và sensors trong MySQL Database
4. Database trả về danh sách dữ liệu cảm biến

5. Backend xử lý và đóng gói dữ liệu thành định dạng JSON để trả về cho FrontEnd
6. FrontEnd nhận dữ liệu và thực hiện hiển thị lên bảng cho người dùng quan sát
7. Người dùng nhập các tiêu chí tìm kiếm trên giao diện

**Bảng 1: Ví dụ các tham số truy vấn dữ liệu cảm biến**

| Tên cảm biến   | Giá trị cảm biến | Khoảng thời gian                          |
|----------------|------------------|-------------------------------------------|
| Cảm biến độ ẩm | 65%              | 08:00:00 18-01-2026 - 10:00:00 19-01-2026 |

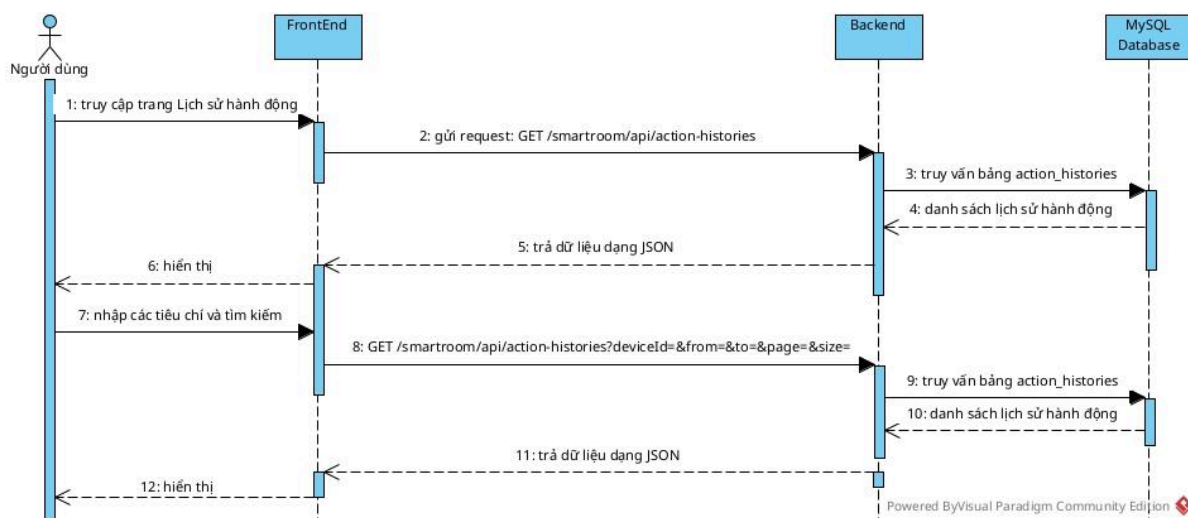
sau đó nhấn nút Tìm kiếm

8. FrontEnd tổng hợp các tiêu chí và gửi yêu cầu API kèm theo các tham số truy vấn:  
name = “doam”, value=”65%”, from=”08:00:00 18-01-2026”, to=”10:00:00 19-01-2026”
9. Backend tiếp nhận yêu cầu và thực hiện truy vấn (Join) giữa hai bảng data\_sensors và sensors trong MySQL Database
10. Database trả về danh sách dữ liệu cảm biến
11. Backend xử lý và đóng gói dữ liệu thành định dạng JSON để trả về cho FrontEnd
12. FrontEnd nhận dữ liệu và thực hiện hiển thị lên bảng cho người dùng quan sát

## e. Luồng xem lịch sử hành động

| ID     | THIẾT BỊ   | HÀNH ĐỘNG | TRẠNG THÁI | THỜI GIAN           |
|--------|------------|-----------|------------|---------------------|
| ACT001 | Máy hút ẩm | BẮT       | ĐANG XỬ LÝ | 09:55:21 15/01/2026 |
| ACT002 | Đèn        | TẮT       | ĐÃ TẮT     | 09:55:23 15/01/2026 |
| ACT003 | Máy hút ẩm | BẮT       | ĐANG XỬ LÝ | 09:55:21 15/01/2026 |
| ACT004 | Quạt       | BẮT       | ĐÃ BẮT     | 09:55:23 15/01/2026 |
| ACT005 | Máy hút ẩm | BẮT       | ĐANG XỬ LÝ | 09:55:21 15/01/2026 |
| ACT006 | Đèn        | TẮT       | ĐÃ TẮT     | 09:55:23 15/01/2026 |
| ACT007 | Máy hút ẩm | BẮT       | ĐANG XỬ LÝ | 09:55:21 15/01/2026 |
| ACT008 | Đèn        | TẮT       | ĐÃ TẮT     | 09:55:23 15/01/2026 |
| ACT009 | Máy hút ẩm | BẮT       | ĐÃ BẮT     | 09:55:21 15/01/2026 |
| ACT010 | Quạt       | TẮT       | ĐÃ BẮT     | 09:55:23 15/01/2026 |
| ACT011 | Máy hút ẩm | BẮT       | ĐÃ BẮT     | 09:55:21 15/01/2026 |
| ACT012 | Đèn        | TẮT       | ĐÃ TẮT     | 09:55:23 15/01/2026 |
| ACT013 | Máy hút ẩm | BẮT       | ĐÃ BẮT     | 09:55:21 15/01/2026 |
| ACT014 | Đèn        | TẮT       | ĐÃ TẮT     | 09:55:23 15/01/2026 |
| ACT015 | Quạt       | BẮT       | ĐÃ BẮT     | 09:55:21 15/01/2026 |

Hình 10: Giao diện trang Lịch sử hành động



Hình 11: Biểu đồ tuần tự luồng xem lịch sử hành động

Chi tiết các bước:

1. Người dùng truy cập vào trang Lịch sử hành động
2. Frontend gửi yêu cầu api GET /smartroom/api/action-histories đến Backend để lấy danh sách tất cả lịch sử hành động kèm phân trang mặc định
3. Backend tiếp nhận yêu cầu và thực hiện truy vấn bảng action\_histories trong MySQL Database
4. Database trả về danh sách lịch sử hoạt động

- Backend xử lý và đóng gói dữ liệu thành định dạng JSON để trả về cho FrontEnd
- FrontEnd nhận dữ liệu và thực hiện hiển thị lên bảng cho người dùng quan sát
- Người dùng nhập các tiêu chí tìm kiếm trên giao diện

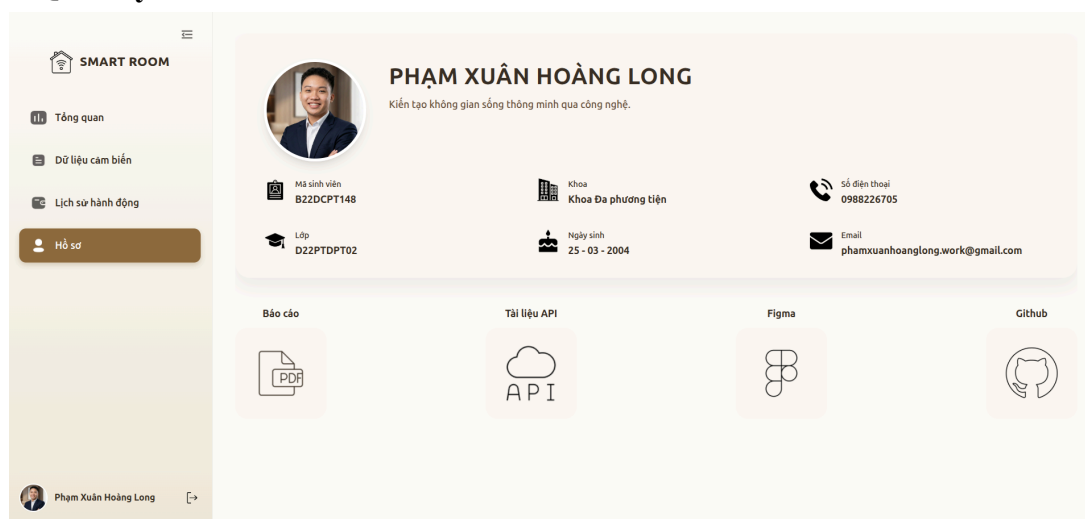
**Bảng 2: Ví dụ các tham số truy vấn lịch sử hành động**

| Tên thiết bị | Khoảng thời gian                          |
|--------------|-------------------------------------------|
| Máy hút ẩm   | 08:00:00 18-01-2026 - 10:00:00 19-01-2026 |

sau đó nhấn Tìm kiếm

- FrontEnd tổng hợp các tiêu chí và gửi yêu cầu API kèm theo các tham số truy vấn:  
deviceId = 1, from="08:00:00 18-01-2026", to="10:00:00 19-01-2026"
- Backend tiếp nhận yêu cầu và thực hiện truy vấn bảng action\_histories trong MySQL Database
- Database trả về danh sách lịch sử hoạt động
- Backend xử lý và đóng gói dữ liệu thành định dạng JSON để trả về cho FrontEnd
- FrontEnd nhận dữ liệu và thực hiện hiển thị lên bảng cho người dùng quan sát

## f. Quản lý hồ sơ



*Hình 12: Giao diện trang Hồ sơ*

## CHƯƠNG III: XÂY DỰNG HỆ THỐNG

### I. Cơ sở dữ liệu

Hệ thống sử dụng MySQL để lưu trữ dữ liệu bền vững. Cấu trúc gồm 4 bảng chính được thiết kế tối ưu cho việc truy vấn lịch sử và quản lý thiết bị:

#### 1. Bảng sensors

- Lưu thông tin các loại cảm biến trong phòng

*Bảng 3: Bảng thuộc tính quan hệ sensors*

| Tên bảng | Tên cột    | Kiểu dữ liệu | Giá trị mặc định  |
|----------|------------|--------------|-------------------|
| sensors  | id         | BIGINT       | Không             |
|          | name       | varchar      | Không             |
|          | created_at | timestamp    | CURRENT_TIMESTAMP |

#### 2. Bảng data\_sensors

- Lưu dữ liệu đo được theo thời gian

*Bảng 4: Bảng thuộc tính quan hệ data\_sensors*

| Tên bảng     | Tên cột    | Kiểu dữ liệu | Giá trị mặc định  |
|--------------|------------|--------------|-------------------|
| data_sensors | id         | BIGINT       | Không             |
|              | sensor_id  | BIGINT       | Không             |
|              | value      | BIGINT       | Không             |
|              | created_at | timestamp    | CURRENT_TIMESTAMP |

#### 3. Bảng devices

- Lưu thiết bị điều khiển trong phòng

**Bảng 5: Bảng thuộc tính quan hệ devices**

| Tên bảng | Tên cột        | Kiểu dữ liệu | Giá trị mặc định  |
|----------|----------------|--------------|-------------------|
| devices  | id             | BIGINT       | Không             |
|          | name           | varchar      | Không             |
|          | current_status | varchar      | ‘OFF’             |
|          | created_at     | timestamp    | CURRENT_TIMESTAMP |

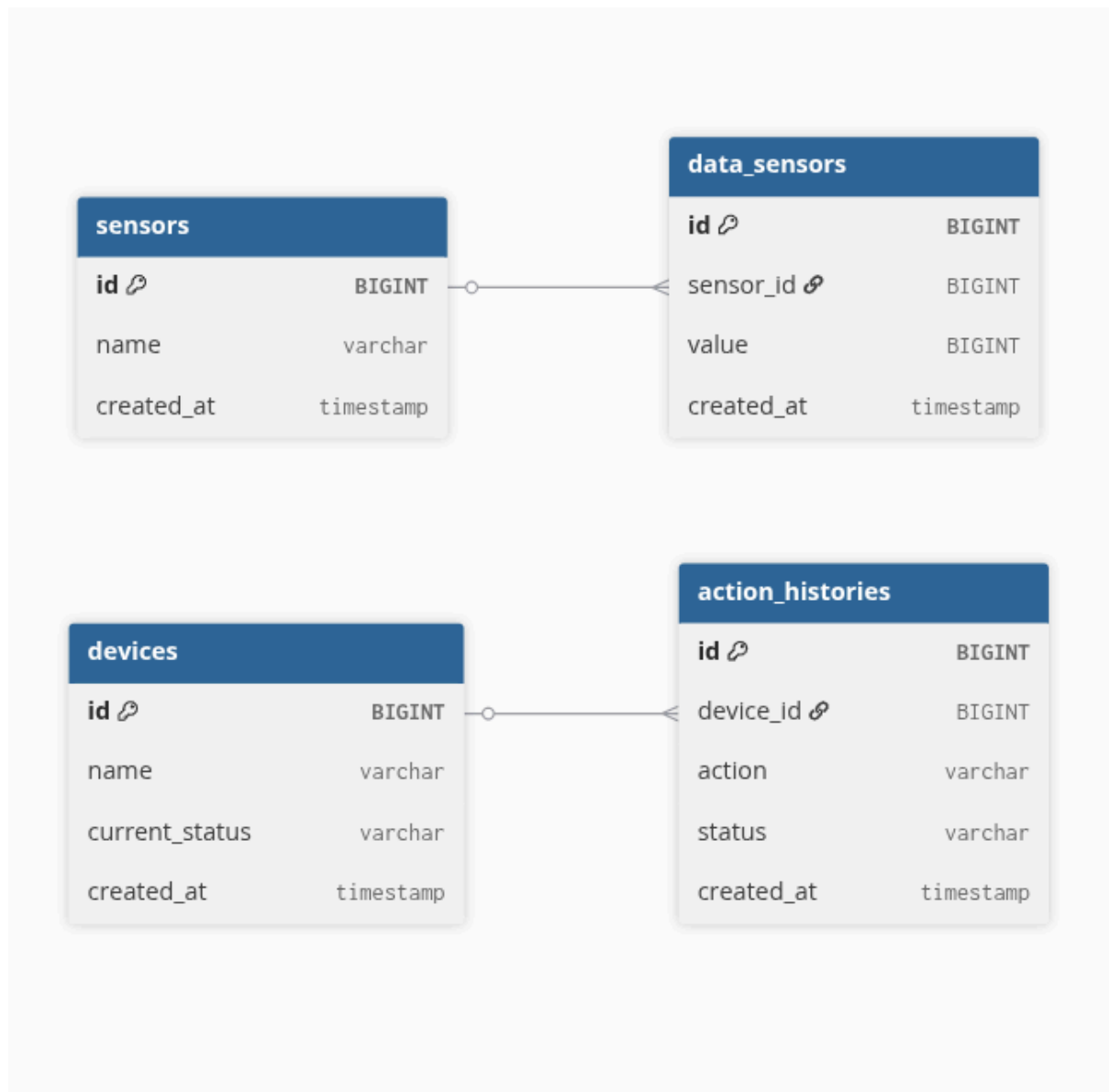
#### 4. Bảng action\_histories

- Lưu lịch sử bật / tắt thiết bị

**Bảng 6: Bảng thuộc tính quan hệ action\_histories**

| Tên bảng         | Tên cột    | Kiểu dữ liệu | Giá trị mặc định  |
|------------------|------------|--------------|-------------------|
| action_histories | id         | BIGINT       | Không             |
|                  | device_id  | BIGINT       | Không             |
|                  | action     | varchar      | Không             |
|                  | status     | varchar      | Không             |
|                  | created_at | timestamp    | CURRENT_TIMESTAMP |

## 5. Biểu đồ E-R



Hình 13: Biểu đồ E-R

## II. Backend

Tầng Backend được phát triển bằng Java 21 kết hợp Spring Boot, đóng vai trò là bộ não trung tâm điều phối toàn bộ luồng dữ liệu giữa phần cứng và giao diện người dùng. Toàn bộ cấu trúc dự án tuân theo mô hình phân lớp chuẩn (Controller – Service – Repository) giúp tách biệt trách nhiệm, dễ kiểm thử và bảo trì về lâu dài.

## 1. Tích hợp MQTT — Giao tiếp với phần cứng

Đây là thành phần kỹ thuật quan trọng và phức tạp nhất của toàn bộ hệ thống, cho phép Backend giao tiếp hai chiều với ESP32 thông qua Mosquitto Broker mà không cần thiết lập kết nối HTTP trực tiếp với phần cứng.

- **MqttConfig.java** — Lớp cấu hình cốt lõi:
  - Lớp này sử dụng thư viện spring-integration-mqtt để thiết lập toàn bộ pipeline xử lý tin nhắn MQTT. Cụ thể, nó khởi tạo hai thành phần:
  - **MqttPahoMessageDrivenChannelAdapter (Inbound)**: Đây là thành phần lắng nghe tự động. Ngay khi ứng dụng khởi động, một kết nối TCP tới Mosquitto Broker được thiết lập và hệ thống đăng ký Subscribe (đăng ký nhận tin) vào Topic smartroom/collect-data. Mỗi khi ESP32 gửi một packet JSON chứa dữ liệu cảm biến, message ngay lập tức được chuyển vào Spring Integration Channel để xử lý tiếp theo. Nếu kết nối bị gián đoạn, cơ chế auto-reconnect sẽ tự động khôi phục kết nối mà không cần can thiệp thủ công.
  - **MqttPahoMessageHandler (Outbound)**: Đây là thành phần dùng để gửi lệnh. Khi người dùng thực hiện thao tác Bật/Tắt thiết bị trên Dashboard, lớp Service sẽ kích hoạt Outbound Handler này để publish một tin nhắn JSON vào Topic smartroom/control. ESP32 — đang Subscribe Topic này — sẽ nhận lệnh và điều chỉnh trạng thái LED tương ứng.
- **MqttServiceImpl.java** — Lớp xử lý nghiệp vụ MQTT: Lớp này đóng vai trò là điểm tiếp nhận trung gian. Mỗi khi Spring Integration Channel nhận được một tin nhắn MQTT, MqttServiceImpl sẽ:
  - Nhận đối tượng Message<?> từ Channel.
  - Trích xuất payload (nội dung chuỗi JSON) từ tin nhắn.
  - Xác định loại tin nhắn (dữ liệu cảm biến hay phản hồi trạng thái thiết bị) dựa trên cấu trúc JSON.
  - Chuyển tiếp xử lý tới MqttUtils.
- **MqttUtils.java** — Trái tim nghiệp vụ: Lớp Utility này chứa toàn bộ logic xử lý dữ liệu thực sự:
  - Hàm processSensorData(String payload):
    - Sử dụng thư viện Jackson (ObjectMapper) để parse chuỗi JSON thành đối tượng Java.
    - Trích xuất các giá trị temperature, humidity, light từ JSON.
    - Tạo các đối tượng DataSensor tương ứng và lưu vào CSDL thông qua DataSensorRepository (Spring Data JPA).
    - Sau khi lưu thành công, gọi SimpMessagingTemplate.convertAndSend("/topic/sensor-data",

data) để broadcast dữ liệu mới nhất tới tất cả các Frontend đang kết nối WebSocket — điều này đảm bảo giao diện người dùng được cập nhật dưới 100ms sau khi cảm biến đo được giá trị mới.

- Hàm `processDeviceStatus(String payload)`:
  - Parse chuỗi JSON phản hồi từ ESP32 sau khi thiết bị đã thực thi lệnh.
  - Cập nhật trường status (ON/OFF) trong bảng devices để phản ánh trạng thái thực tế của phần cứng.
  - Tạo một bản ghi mới vào bảng `action_histories` bao gồm: `device_id`, action (ON/OFF) và `action_at` (timestamp hiện tại) — tạo thành nhật ký kiểm soát toàn diện.

## 2. Thiết kế REST API

Backend cung cấp đầy đủ bộ API RESTful phục vụ request tương tác từ Frontend. Tất cả các API đều trả về dữ liệu định dạng JSON.

| Phương thức | Endpoint                                           | Mô tả                                                          |
|-------------|----------------------------------------------------|----------------------------------------------------------------|
| GET         | <code>/smartroom/api/sensors</code>                | Lấy lịch sử dữ liệu cảm biến (phân trang, lọc theo thời gian). |
| GET         | <code>/smartroom/api/sensors/names</code>          | Lấy danh sách tên các loại cảm biến (phục vụ bộ lọc).          |
| GET         | <code>/smartroom/api/device</code>                 | Lấy danh sách và trạng thái hiện tại của tất cả thiết bị.      |
| POST        | <code>/smartroom/api/device/control</code>         | Gửi lệnh Bật/Tắt thiết bị (JSON payload).                      |
| GET         | <code>/smartroom/api/action-histories</code>       | Lấy nhật ký điều khiển thiết bị (phân trang).                  |
| GET         | <code>/smartroom/api/action-histories/names</code> | Lấy danh sách tên thiết bị có trong lịch sử (phục vụ bộ lọc).  |

Mỗi lớp Controller được chú thích bằng `@RestController` và `@RequestMapping`, xử lý việc nhận request và trả về response. Logic nghiệp vụ được ủy thác hoàn toàn xuống lớp Service, và lớp Service tương tác với cơ sở dữ liệu thông qua lớp Repository — đảm bảo tính tách biệt và dễ kiểm thử cho từng lớp.

### 3. WebSocket & Giao thức STOMP — Cập nhật Real-time

Để giải quyết bài toán đẩy dữ liệu cảm biến liên tục lên Frontend mà không gây tốn tài nguyên server, hệ thống triển khai WebSocket với giao thức STOMP chạy bên trên:

- **Cấu hình:**  
Lớp WebSocketConfig đăng ký endpoint /ws cho phép Frontend khởi tạo kết nối SockJS. Thêm vào đó, một Message Broker đơn giản được thiết lập với tiền tố /topic cho các kết nối Pub/Sub.
- **Cơ chế hoạt động:**  
Khi MqttUtils.processSensorData() nhận được dữ liệu mới từ ESP32, nó gọi tới SimpMessagingTemplate.convertAndSend("/topic/sensor-data", payload), kích hoạt việc gửi dữ liệu tới tất cả các client đang Subscribe vào Topic /topic/sensor-data.
- **Kết quả:**  
Dashboard của người dùng được cập nhật liên tục mà không cần F5. Bảng thông tin nhỏ hơn rất nhiều so với kỹ thuật Polling HTTP truyền thống.

## III. Frontend

Giao diện người dùng được xây dựng bằng ReactJS 19, áp dụng kiến trúc Component-based hiện đại. Toàn bộ UI Components được xây dựng trên nền tảng Material UI (MUI) để đảm bảo tính nhất quán về thiết kế. Dữ liệu được trực quan hóa bằng Recharts — thư viện biểu đồ được tối ưu hóa chuyên biệt cho ứng dụng React. Kết nối WebSocket được thực hiện thông qua cặp thư viện SockJS + @stomp/stompjs.

### 1. Quản lý kết nối WebSocket

Khi ứng dụng khởi động, một useEffect hook được gọi một lần duy nhất để thiết lập kết nối STOMP tới Backend:

- Tạo một SockJS client kết nối vào http://[server]/ws.
- Khởi tạo StompClient bọc bên ngoài SockJS.
- Sau khi kết nối thành công, Subscribe vào Topic /topic/sensor-data.
- Mỗi khi nhận được message, cập nhật State React (useState) — React tự động re-render component để hiển thị giá trị mới.
- Hàm cleanup trong useEffect sẽ ngắt kết nối WebSocket khi component bị unmount, tránh rò rỉ bộ nhớ (memory leak).

### 2. Trang tổng quan

Đây là màn hình trung tâm và quan trọng nhất của ứng dụng, nơi người dùng có thể vừa giám sát vừa điều khiển toàn bộ hệ thống:

- Cards thông số môi trường: Ba Card lớn hiển thị nổi bật giá trị Nhiệt độ (°C), Độ ẩm (%) và Cường độ ánh sáng (lux) hiện tại. Các giá trị này được cập nhật thời gian thực từ WebSocket mà không cần tải lại trang. Mỗi Card có màu sắc và icon riêng để người dùng phân biệt dễ dàng.
- Biểu đồ đường lịch sử (LineChart): Sử dụng <LineChart> từ Recharts để trực quan hóa sự biến động của nhiệt độ và độ ẩm theo thời gian trong ngày. Dữ liệu được lấy từ REST API /api/data-sensors và được lọc theo khoảng thời gian người dùng chọn. Trục X hiển thị thời gian, trục Y hiển thị giá trị đo. Người dùng có thể hover vào điểm bất kỳ trên biểu đồ để xem chi tiết giá trị tại thời điểm đó (Tooltip).
- Panel điều khiển thiết bị: Ba Switch gạt (MUI <Switch>) tương ứng với Quạt, Đèn, Máy hút ẩm. Trạng thái Switch được đồng bộ với dữ liệu lấy từ API /api/devices. Khi người dùng gạt Switch, một request HTTP POST được gửi tới /api/devices/{id}/toggle. Trong thời gian chờ phản hồi, Switch hiển thị trạng thái loading để ngăn người dùng thao tác nhiều lần liên tiếp (debounce). Sau khi nhận phản hồi thành công từ Server, trạng thái Switch được cập nhật chính thức.

### 3. Trang Dữ liệu cảm biến

Đây là màn hình cho phép người dùng tra cứu và phân tích toàn bộ lịch sử dữ liệu môi trường đã được hệ thống ghi nhận:

- Bảng dữ liệu có phân trang: Sử dụng <DataGrid> của MUI để hiển thị dữ liệu dạng bảng với các cột: STT, Tên cảm biến, Giá trị, Đơn vị, Thời gian ghi nhận. Thành phần này hỗ trợ phân trang (Pagination) phía Server — mỗi lần chỉ tải một trang (ví dụ: 20 bản ghi), giúp ứng dụng vận hành mượt mà dù cơ sở dữ liệu chứa hàng chục nghìn bản ghi.
- Bộ lọc dữ liệu: Người dùng có thể lọc theo loại cảm biến (Nhiệt độ / Độ ẩm / Ánh sáng) và theo khoảng thời gian (từ ngày...đến ngày...). Khi thay đổi bộ lọc, một request API mới được gửi đi với các tham số tương ứng.

### 4. Trang Lịch sử hoạt động

Đây là màn hình kiểm soát, ghi lại toàn bộ nhật ký mọi lệnh điều khiển đã được thực hiện:

- Bảng nhật ký: Hiển thị đầy đủ các cột: Tên thiết bị, Thao tác thực hiện (BẬT/TẮT), Kết quả, Thời điểm thực hiện. Dữ liệu được lấy từ bảng action\_histories thông qua API /api/action-histories.
- Phân trang và tìm kiếm: Hỗ trợ phân trang theo Server, cho phép tra cứu nhanh lịch sử điều khiển của một thiết bị cụ thể.

- Màu sắc phân biệt trạng thái: Các tag trạng thái BẬT/TẮT được tô màu khác nhau (xanh/đỏ) giúp người dùng nhận diện nhanh mà không cần đọc văn bản.

## **5. Trang Hồ sơ người dùng**

- Hiển thị thông tin tài khoản hiện tại (họ tên, email, ngày tạo tài khoản).
- Form chỉnh sửa thông tin cá nhân sử dụng MUI `<TextField>` với validation phía client (kiểm tra trường rỗng, định dạng email hợp lệ) trước khi cho phép gửi request lên Server.
- Nút Lưu thay đổi chỉ được kích hoạt khi người dùng thực sự thay đổi nội dung, tránh gửi request thừa khi không có gì thay đổi.
- Sau khi cập nhật thành công, một Snackbar (thông báo nhỏ góc màn hình) xuất hiện xác nhận thao tác đã hoàn thành.

## CHƯƠNG IV: KẾT LUẬN VÀ ĐÁNH GIÁ

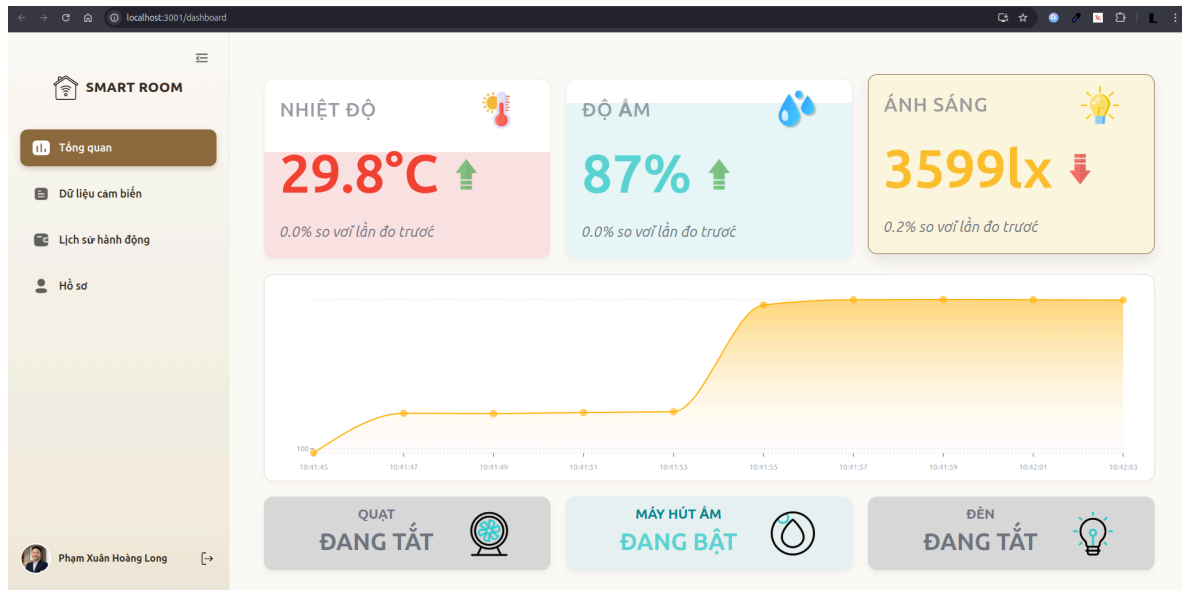
### I. Kết quả đạt được

#### 1. Kết quả

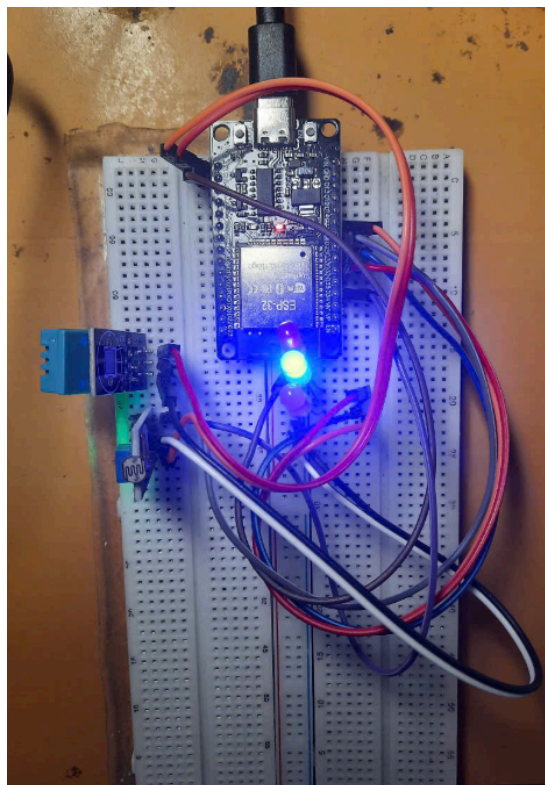
Sau một thời gian nghiên cứu và triển khai, dự án Smart Room IoT đã hoàn thành đầy đủ các mục tiêu kỹ thuật và ứng dụng đề ra ban đầu. Những kết quả cụ thể bao gồm:

- Về mặt kiến trúc hệ thống: Đã xây dựng thành công mô hình IoT 4 tầng tiêu chuẩn (Perception, Network, Service, Application). Việc phân tầng rõ rệt giúp hệ thống vận hành ổn định, dễ dàng mở rộng và bảo trì từng phần độc lập mà không ảnh hưởng đến toàn cục.
- Về mặt phần cứng (Hardware): Thiết kế và lắp đặt hoàn chỉnh module điều khiển trung tâm dựa trên ESP32. Hệ thống đã chứng minh khả năng thu thập dữ liệu chính xác từ các cảm biến nhiệt độ, độ ẩm (DHT11) và ánh sáng (LDR), đồng thời thực thi lệnh điều khiển các thiết bị chấp hành (LED mô phỏng Quạt, Đèn, Máy hút ẩm) với độ tin cậy cao.
- Về mặt truyền thông và xử lý dữ liệu:
  - Triển khai thành công giao thức MQTT cho việc giao tiếp giữa phần cứng và Backend, đảm bảo băng thông tối ưu và khả năng chịu lỗi tốt.
  - Xây dựng hệ thống Backend bằng Spring Boot xử lý luồng dữ liệu phức tạp, lưu trữ bền vững vào MySQL với cấu trúc 4 bảng tối ưu cho truy vấn lịch sử.
  - Ứng dụng công nghệ WebSocket (STOMP) giúp giải quyết triệt để bài toán cập nhật dữ liệu thời gian thực (Real-time), mang lại trải nghiệm mượt mà cho người dùng cuối.
- Về mặt giao diện người dùng (UI/UX): Phát triển một Dashboard hiện đại bằng ReactJS, tích hợp các biểu đồ trực quan (Recharts) và các thành phần giao diện chuẩn mực (Material UI). Hệ thống hỗ trợ hiển thị tốt trên nhiều loại thiết bị (Responsive), giúp người dùng có thể giám sát và điều khiển phòng ở bất cứ đâu.

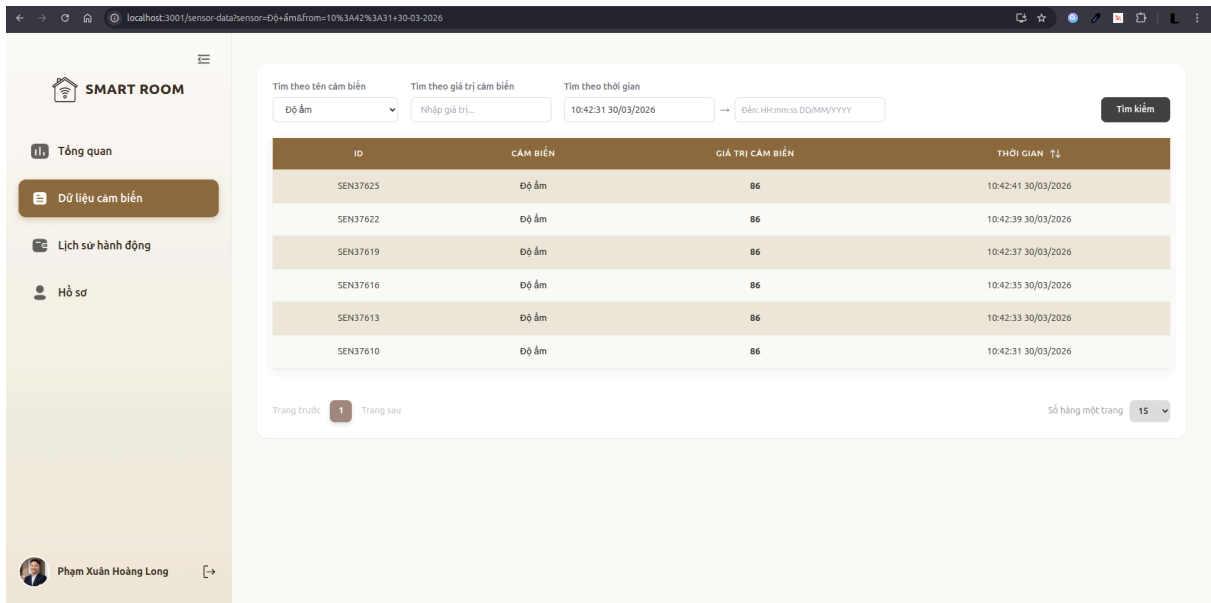
## 2. Một số hình ảnh kết quả đạt được



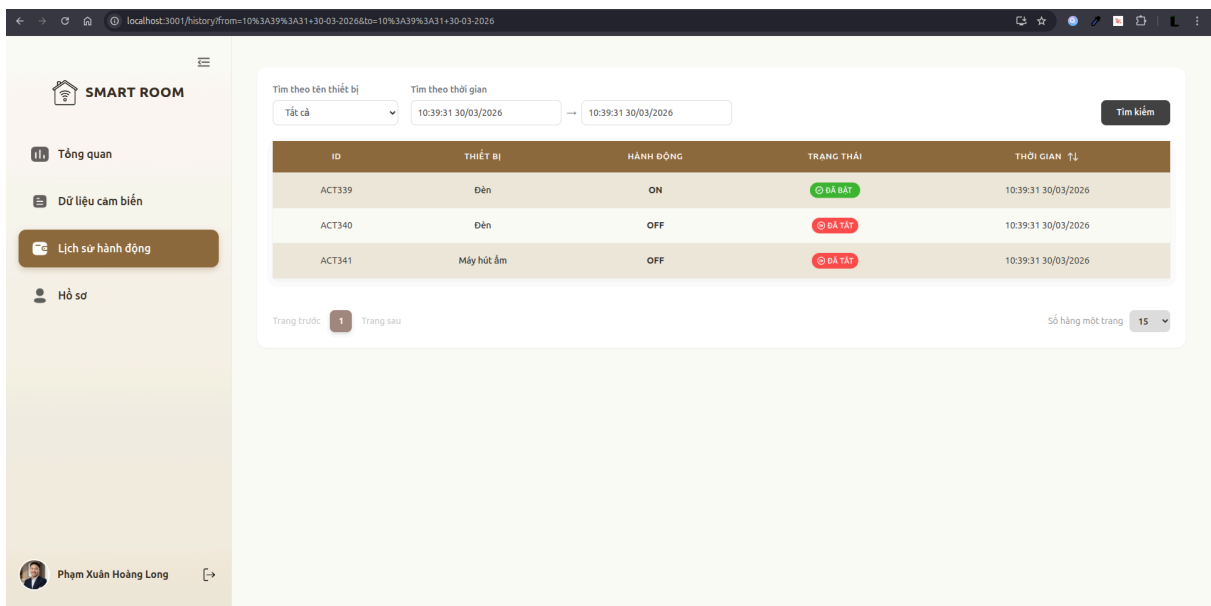
Hình 14: Trang tổng quan



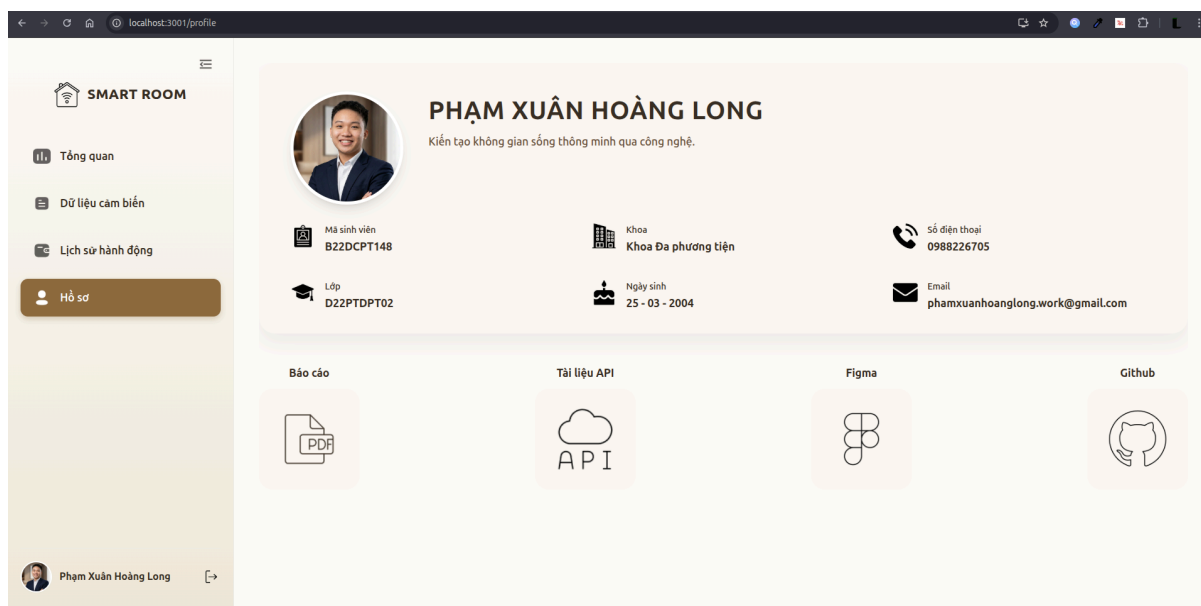
Hình 15: Điều khiển bật đèn qua giao diện web



Hình 16: Tìm kiếm dữ liệu cảm biến



Hình 17: Tìm kiếm lịch sử hành động



Hình 18: Trang hồ sơ

## II. Đánh giá ưu và nhược điểm

### 1. Ưu điểm

- Tính thực tiễn cao: Hệ thống giải quyết được nhu cầu thực tế về giám sát và tự động hóa không gian sống, có khả năng áp dụng trực tiếp vào mô hình nhà thông minh hoặc văn phòng thông minh.
- Hiệu năng ổn định: Nhờ sự kết hợp giữa MQTT và WebSocket, độ trễ từ khi cảm biến ghi nhận dữ liệu đến khi hiển thị trên màn hình người dùng là cực thấp.
- Dữ liệu minh bạch: Hệ thống cung cấp nhật ký hoạt động (Action History) và lịch sử cảm biến chi tiết, giúp người dùng dễ dàng đối soát và theo dõi xu hướng thay đổi môi trường.
- Kiến trúc hiện đại: Sử dụng các công nghệ mới nhất (Java 21, React 19) giúp hệ thống có khả năng bảo mật tốt và dễ dàng tích hợp các dịch vụ đám mây (Cloud Services) trong tương lai.

### 2. Nhược điểm

- Phụ thuộc vào kết nối mạng: Do sử dụng mô hình Client-Server dựa trên Wi-Fi, nếu đường truyền internet hoặc mạng nội bộ gặp sự cố, việc điều khiển và giám sát sẽ bị gián đoạn.
- Mức độ tự động hóa còn cơ bản: Hiện tại hệ thống chủ yếu dựa trên tương tác trực tiếp của người dùng (Manual control). Việc tự động hóa theo kịch bản

(Automation rules) vẫn còn đơn giản, chưa tích hợp được các thuật toán AI/ML để học hỏi thói quen người dùng.

- Khả năng bảo mật: Tuy đã có các bước xác thực cơ bản nhưng hệ thống vẫn cần được mã hóa sâu hơn (TLS/SSL cho MQTT và HTTPS cho nội dung Web) để đảm bảo an toàn tuyệt đối trước các cuộc tấn công mạng.

### **III. Kết luận**

Hệ thống Smart Room IoT đã chứng minh được tính khả thi và tiềm năng ứng dụng mạnh mẽ của việc kết hợp các công nghệ mã nguồn mở trong xây dựng hệ sinh thái nhà thông minh. Thông qua sự kết nối tinh tế giữa phần cứng nhúng năng suất cao (ESP32) và kiến trúc phần mềm phân tầng hiện đại (Spring Boot - ReactJS), hệ thống không chỉ giải quyết triệt để bài toán giám sát và điều khiển thiết bị truyền thống, mà còn kiến tạo một nền tảng dữ liệu số đồng bộ, bảo mật và hiệu quả.

Sự thành công của dự án này không chỉ dừng lại ở các kết quả kỹ thuật đạt được mà còn mở ra những cơ sở thực tiễn quan trọng cho việc nghiên cứu và phát triển các giải pháp IoT phức tạp hơn. Trong kỷ nguyên chuyển đổi số, đây chính là bước đệm cần thiết để hướng tới mục tiêu tối ưu hóa trải nghiệm người dùng, tiết kiệm năng lượng và nâng cao chất lượng cuộc sống thông qua những giá trị thực chất mà công nghệ mang lại.

## TÀI LIỆU THAM KHẢO

1. **Espressif Systems** - ESP32 Technical Reference Manual:  
*<https://www.espressif.com/en/support/documents/technical-documents>*
2. **MQTT.org** - MQTT Essentials & Specification:  
*<https://mqtt.org/getting-started/>*
3. **Spring.io** - Spring Boot Reference Documentation:  
*<https://docs.spring.io/spring-boot/docs/current/reference/html/>*
4. **MySQL.com** - MySQL Reference Manual:  
*<https://dev.mysql.com/doc/refman/8.0/en/>*
5. **MUI.com** - Material UI Documentation:  
*<https://mui.com/material-ui/getting-started/>*
6. **Recharts.org** - Recharts API Reference: *<https://recharts.org/en-US/api>*
7. **STOMP.github.io** - STOMP Protocol Specification:  
*<https://stomp.github.io/stomp-specification-1.2.html>*